C7 Real-Time Intelligence

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track C: Microsoft Fabric 2
C7 Real-Time Intelligence 3
C7-L1 Eventstream, events in, routed out 3
C7-L2 Eventhouse and KQL for time-series questions 7
C7-L3 Activator, rules that act on events 11
Track C best-practice checklist 16
Appendix: solutions and answer keys 17

Track C: Microsoft Fabric

Unify data, analytics and Copilot in Fabric, and verify what Copilot produces.

C7 Real-Time Intelligence

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Eventstream, Eventhouse, KQL and Activator.

C7-L1 Eventstream, events in, routed out

Concept

An **eventstream** brings real-time events into Fabric, optionally transforms them, and routes them to destinations, without code. It is Microsoft's recommended path for low-latency, event-driven ingestion (lesson C4-L1).

Sources include Azure Event Hubs, IoT Hub, custom apps, and change data capture (CDC) connectors for Azure SQL Database, PostgreSQL, MySQL and SQL Server on a VM. A CDC source first captures a snapshot of the current data, then streams the changes.

Transformations are no-code operators on the stream: filter, manage fields, aggregate over a time window, group by, union, join, expand. You can split one stream into **derived streams** for different consumers.

Destinations (Microsoft Learn, checked September 2026):

- **Eventhouse**, for KQL analysis (lesson C7-L2). Two ingestion modes: **direct ingestion**, or **event processing before ingestion** when you apply transformations in the eventstream first.
- **Lakehouse**. Events are converted to Delta Lake format and written to lakehouse tables.
- **Activator**, to trigger actions on conditions (lesson C7-L3).
- A **custom endpoint**, to route events to your own application.

Writing to an eventhouse, lakehouse or Activator destination needs **Contributor or higher** on the destination's workspace.

Design choices that matter:

- **Filter early.** Drop events nobody needs in the eventstream, before they cost storage and query time.
- **Keep the raw stream.** Route an unfiltered copy to an eventhouse or lakehouse table as your bronze layer, and derive cleaned streams from it. You cannot replay events you dropped.

- **Decide the event time.** Events carry their own timestamp. Use it for windows and analysis, not the time of arrival, which drifts when senders buffer or reconnect.

Why it matters

A fleet team filtered its eventstream to keep only engine-fault events and wrote those to an eventhouse. Three months later, engineers needed the normal readings before each fault to build a failure model. They had never been stored. Keeping the raw stream in a cheap table, and deriving the fault stream from it, would have preserved the history the model needed.

Worked example: weak code

```
1 Source: IoT Hub (all vehicle telemetry)
2 Transform: Filter event_type == "fault"
3 Destination: Eventhouse table faults
```

1. **Line 2.** The only copy of the data is filtered. Normal readings are lost for good.

Worked example: strong code

```
1 Source: IoT Hub (all vehicle telemetry)
2 Destination 1: Eventhouse table telemetry_raw        (direct ingestion,
    every event)
3 Derived stream: faults = Filter event_type == "fault"
4 Destination 2: Activator rule on faults               (alert within
    seconds)
5 Destination 3: Lakehouse table bronze.vehicle_faults  (Delta, for Spark
    models)
```

1. **Line 2.** The raw stream is kept as bronze. Anything else can be rebuilt from it.
2. **Line 3.** Consumers get a filtered stream without losing the source.
3. **Line 4.** Alerts come from the stream, not from a scheduled query.

Land the raw stream once, derive filtered streams for consumers, and route each to the destination that suits it.

Recap card

- **What an eventstream does** Ingests real-time events, transforms them without code, and routes them to destinations.
- **Eventstream destinations** Eventhouse (direct or processed ingestion), Lakehouse (Delta tables), Activator, custom endpoint.
- **Permission for eventhouse, lakehouse or Activator destinations** Contributor or higher on the destination workspace.
- **Protect replay** Land the raw stream first. Derive filtered streams from it.

Practice

Exercise 1. Multiple choice (level 1)

An eventstream sends events to a lakehouse destination. In what format are they stored?

- a) JSON files under Files
- b) Delta Lake tables
- c) CSV files
- d) KQL tables

Why?

- r1) The lakehouse destination converts real-time events to Delta Lake format in lakehouse tables.
- r2) Eventstream only writes Delta.

Exercise 2. Multiple choice (level 2)

Which permission does the person setting up an eventstream need on the workspace of an eventhouse destination?

- a) Viewer
- b) Contributor or higher
- c) Admin only
- d) None: eventstreams write as the service

Why?

- r1) Microsoft Learn requires Contributor or higher on the destination workspace for eventhouse, lakehouse and Activator destinations.
- r2) Contributors can create any item in any workspace.

Exercise 3. Put in order (level 2)

Order the design steps for a new telemetry eventstream.

- Connect the source
- Connect each derived stream to its destination
- Create derived streams with filters for each consumer
- Route every event to a raw table
- Compare event counts at source and destinations

Exercise 4. Write the function (level 3)

Write `route(event: dict[str, object]) -> list[str]` modelling the strong design. Every event goes to `"telemetry_raw"`. Events with `event_type == "fault"` also go to `"activator_faults"` and `"bronze.vehicle_faults"`. Events without an `event_type` key go to `"telemetry_raw"` and `"quarantine"`. Return destinations in that order.

Your code must pass these checks:

- Every event is kept in telemetry_raw.
- Faults go to the raw table, Activator and the bronze faults table, in that order.
- Events without event_type are kept raw and quarantined, not dropped or crashing.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Destinations for one telemetry event. Every event is kept raw.
def route(event: dict[str, object]) -> list[str]: ...
```

Starter code:

```python
def route(event):
    if event["event_type"] == "fault":
        return ["activator_faults"]
    return []
```

Exercise 5. Architecture design (level 4)

Stream order changes from an Azure SQL database into Fabric for a live dashboard and for nightly Spark models.

Options: Azure SQL Database CDC source, A pipeline polling the table every minute.

1. Source connector
2. Destination for the live dashboard
3. Destination for nightly Spark models

Sources

- Eventstreams ingest, transform without code and route events; CDC source connectors for Azure SQL, PostgreSQL, MySQL and SQL Server on VM capture a snapshot then changes. (verified: https://learn.microsoft.com/en-us/fabric/real-time-intelligence/event-streams/overview (updated 2026-04-29))

- Destinations include custom endpoint, eventhouse (direct ingestion or processing before ingestion), lakehouse (Delta) and Activator; Contributor or higher is required for eventhouse, lakehouse and Activator destinations. (verified: https://learn.microsoft.com/en-us/fabric/real-time-intelligence/event-streams/add-manage-eventstream-destinations (updated 2026-03-19))

C7-L2 Eventhouse and KQL for time-series questions

Concept

An **eventhouse** holds one or more **KQL databases**, built for high-volume, time-stamped data such as telemetry, logs and clickstreams. You query them with the **Kusto Query Language (KQL)**: a pipeline of operators joined by `|`, read top to bottom.

```
telemetry_raw
| where event_time > ago(1h)
| where event_type == "reading"
| summarize avg_rpm = avg(rpm), n = count() by vehicle_id, bin(event_time, 5m)
```

- Put `where` filters first, especially on time. They cut the data every later operator touches.
- `ago(1h)` is one hour before now. `bin(event_time, 5m)` rounds times **down** to five-minute buckets.
- `summarize ... by` aggregates per group.

Three KQL behaviours catch people out (Microsoft Learn, checked September 2026):

- `take` **returns arbitrary rows.** There is no guarantee which records come back, and repeated runs can differ, unless the data is sorted. For "the latest 10", use `top 10 by event_time desc`.
- `summarize` **leaves gaps.** A five-minute bin with no events produces no row, so a chart draws a straight line across an outage. `make-series ... default = 0 ... step 5m` fills missing bins.
- **Empty input.** `summarize count()` without `by` returns one row. With `by`, empty input returns no rows at all.

Data policies are set per database or table. The **retention policy** removes old data automatically. The default is 3,650 days, the maximum 36,500, the minimum one day, and you can set it to unlimited. The **caching policy** decides how much recent data stays in the hot cache on local SSD: faster

queries, higher cost. Set retention from the use: raw telemetry kept 90 days, daily aggregates kept for years.

Turn on **OneLake availability** for a database or table to expose its data as Delta in OneLake, so Spark and the SQL analytics endpoint can read it without a copy.

Why it matters

An on-call dashboard showed the "latest readings" from a KQL query ending in `| take 10`. During an incident, the panel showed readings from two hours earlier, mixed with current ones, and the engineer concluded the sensor had recovered. It had not. `take` returned arbitrary rows. `top 10 by event_time desc` would have shown the true latest readings.

Worked example: weak code

```
telemetry_raw
| summarize avg_rpm = avg(rpm) by vehicle_id, bin(event_time, 5m)
| where event_time > ago(1h)
| take 10
```

1. **Line 2.** Aggregates the whole table before filtering to the last hour.
2. **Line 4.** Arbitrary rows, different on each run.

Worked example: strong code

```
telemetry_raw
| where event_time > ago(1h)
| where event_type == "reading"
| make-series avg_rpm = avg(rpm) default = 0 on event_time step 5m by
  vehicle_id

telemetry_raw
| where event_time > ago(15m)
| top 10 by event_time desc
```

1. **Line 2.** Time filter first: less data for every later step.
2. **Line 4.** Every five-minute bin exists, so outages show as zeros, not as smooth lines.
3. **Line 8.** The true latest ten, every time.

Filter on time first, use top for "latest", make-series for continuous charts, and set retention and caching per table on purpose.

Recap card

- **take vs top in KQL** take: arbitrary rows, not repeatable. top N by col desc: ordered, repeatable.

- **Gaps in time series** summarize omits empty bins. make-series ... default = 0 fills them.
- **KQL retention policy defaults** Default 3,650 days, maximum 36,500, minimum 1 day, or unlimited.
- **Caching policy** How much recent data stays in the hot SSD cache: faster queries, higher cost.

Practice

Exercise 1. Multiple choice (level 1)

A query ends with `| take 10`. What do you get?

- a) The 10 most recent rows
- b) The first 10 rows ingested
- c) 10 arbitrary rows, possibly different on each run
- d) An error unless the table is sorted

Why?

- r1) take guarantees nothing about which records it returns unless the data is sorted.
- r2) take samples randomly with a fixed seed.

Exercise 2. Multiple choice (level 2)

A sensor sent nothing between 10:05 and 10:20. What does `summarize n = count() by bin(event_time, 5m)` return for those bins?

- a) Rows with n = 0
- b) No rows for those bins
- c) Rows with n = null
- d) An error

Why?

- r1) summarize groups existing rows. Bins with no rows do not appear. make-series fills them.
- r2) KQL drops zero counts to save space.

Exercise 3. Fill the gap (level 2)

Return the 10 most recent readings from the last hour.

```
1 telemetry_raw
2 | where event_time > _____(1)(1h)
3 | _____(2) 10 by event_time desc
```

Exercise 4. Write the function (level 3)

Write a KQL query on `telemetry_raw` that returns, for the last 24 hours, the average `rpm` per `vehicle_id` in one-hour steps, with **every** hour present and missing hours as 0. Filter on time first. Name the series `avg_rpm`.

Shape of the answer. The reference answer follows this outline. Replace each ... with your own code.

```
telemetry_raw
| where ...
| make-series ...
```

Starter code:

```
telemetry_raw
| summarize avg_rpm = avg(rpm) by vehicle_id, bin(event_time, 1h)
```

Exercise 5. Architecture design (level 4)

Set data policies for a telemetry eventhouse.

Options: 90 days, Leave the default of 3,650 days, 7 days.

1. Retention for telemetry_raw (queried for 30 days, needed for audits for 90)
2. Hot cache for telemetry_raw
3. Retention for daily aggregates used in yearly trends

Sources

- take guarantees nothing about which records it returns unless the data is sorted; bin rounds down; summarize on empty input with a by clause returns no rows; make-series assigns a default to absent values. (verified: https://learn.microsoft.com/en-us/kusto/query/ (take-operator, bin-function, summarize-operator, make-series-operator; view=microsoft-fabric))
- KQL database retention defaults to 3,650 days, maximum 36,500, minimum one day, unlimited option; caching policy controls data kept in the hot cache. (verified: https://learn.microsoft.com/en-us/fabric/real-time-intelligence/data-policies (updated 2026-03-22))
- OneLake availability can be enabled per database or table. (verified: https://learn.microsoft.com/en-us/fabric/real-time-intelligence/eventhouse (updated 2026-06-15))

C7-L3 Activator, rules that act on events

Concept

Activator is Fabric's no-code event detection engine. It watches data and takes an action when a condition is met. Its sources include eventstreams, Fabric and Azure events, Power BI reports and Real-Time dashboards.

Activator groups **events** into **objects** by a shared ID, such as `vehicle_id`. It evaluates each rule **per object** and keeps state for each one. "Vehicle 17 overheated" is a separate fact from "vehicle 42 overheated".

Two kinds of condition behave differently (Microsoft Learn, checked September 2026):

- **Change conditions** fire on a transition. `Increases above 100` fires when a value crosses 100 from below. `Becomes true` fires when a boolean changes from false to true.
- **State conditions** fire on every event that matches. `Is greater than 100` fires for each reading above 100.

For alerts, you want a change condition, or a state condition with an **occurrence** setting: "When it has been true for 10 minutes" fires only if the condition holds for the whole period. **No presence of data** (heartbeat) fires when no new events arrive within a set time. Use it to detect a silent sensor.

Actions include an email, a Teams message, a Power Automate flow, or running a Fabric item such as a pipeline or notebook.

Limits to design around:

- Up to 10,000 incoming events per second per rule. Above that, Activator stops the rule.
- Emails and Teams messages: 30 per rule per recipient per hour.
- Late events: by default, Activator drops events that arrive after their time window closes. The **Wait time for late-arriving events** setting (default two minutes) lets them in, at the cost of added latency.

Why it matters

A plant set a rule `Temperature Is greater than 80` on a stream that reports every 10 seconds. One oven ran hot for 20 minutes. The rule tried to send 120 emails to the shift lead. After 30, the rate limit throttled the rest, and the lead muted the alerts. An `Increases above 80` rule, or `Is greater than 80` with an occurrence of five minutes, would have sent one useful alert.

Worked example: weak code

```
1 Object: all events (no ID)
2 Condition: temperature Is greater than 80
3 Occurrence: Every time the condition is met
4 Action: Email shift lead
```

1. **Line 1.** One oven's reading clears the state of another. Group by oven.
2. **Line 2.** A state condition fires on every hot reading.

Worked example: strong code

```
1 Object: oven, keyed by oven_id
2 Condition: temperature Is greater than 80
3 Occurrence: When it has been true for 5 minutes
4 Action: Teams message to the line channel
5 Rule 2: No presence of data for 2 minutes per oven_id
```

1. **Line 1.** State is tracked per oven.
2. **Line 3.** One alert for a sustained problem, not one per reading.
3. **Line 5.** A silent sensor is a problem too. Heartbeat detects it.

Key each rule by the object it describes. Fire on change or sustained state, not on every event. Add a heartbeat for silence.

Recap card

- **Events and objects in Activator** Events are grouped into objects by a shared ID. Rules run and keep state per object.
- **Increases above vs Is greater than** Increases above fires on the crossing. Is greater than fires on every matching event.
- **Detect a silent sensor** No presence of data (heartbeat) fires when no events arrive within a set time.
- **Activator limits to design around** 10,000 events per second per rule. 30 emails or Teams messages per rule per recipient per hour.

Practice

Exercise 1. Multiple choice (level 1)

A stream reports `temperature` every 10 seconds. It rises above 100 and stays there for 3 minutes. How many times does `Increases above 100` fire?

- a) Once
- b) 18 times, once per reading
- c) Never, because the value never decreases
- d) Once every minute

Why?

- r1) Increases above is a change condition. It fires when the value crosses the threshold from below.
- r2) Activator sends one alert per rule per day.

Exercise 2. Multiple choice (level 2)

A sensor stops sending data. Which Activator condition detects it?

- a) Is less than 1
- b) No presence of data
- c) Becomes false
- d) Changes

Why?

- r1) No presence of data fires when no new events arrive within a set time. Other conditions need an event to evaluate.
- r2) A silent sensor sends zeros.

Exercise 3. Fill the gap (level 2)

Alert once when a freezer stays above -15 for 10 minutes. Complete the rule.

```
Object: freezer, keyed by _____(1)
Condition: temperature Is greater than -15
Occurrence: When it has been _____(2) for 10 minutes
```

Exercise 4. Write the function (level 3)

Model an Activator `Increases above` rule. Write `activations(events: list[tuple[str, float]], threshold: float) -> list[int]`. Each event is `(object_id, value)` in arrival order. Return the indexes of events where that object's value goes above `threshold` and its previous value was at or below it. An object's first event counts as crossing if it is above the threshold. Track state per object.

Your code must pass these checks:

- Fire on each crossing from at or below to above, not on every hot reading.
- Track state per object. Oven b's cool reading does not reset oven a.
- A value equal to the threshold is not above it.
- No events, no activations.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Indexes of events where an object's value crosses above a threshold.
```

```
def activations(
    events: list[tuple[str, float]],
    threshold: float,
) -> list[int]: ...
```

Starter code:

```
def activations(events, threshold):
    return [i for i, (_, value) in enumerate(events) if value > threshold]
```

Exercise 5. Architecture design (level 4)

Design alerts for 400 delivery vans that send GPS and engine data every 5 seconds. The fleet manager wants one Teams message when a van's engine runs hot for a sustained period, and to know when a van goes silent.

Options: van_id, One object for the whole fleet.

1. Object key
2. Hot-engine condition
3. Silent van

Sources

- Activator is a no-code event detection engine; events are grouped into objects by a shared identifier; rules are evaluated per object with state tracking; subsecond latency for stateless rules on streaming data. (verified: https://learn.microsoft.com/en-us/fabric/real-time-intelligence/data-activator/activator-introduction (updated 2026-04-17))
- Conditions include Increases above, Decreases below, Is greater than, Becomes true/false, Changes and No presence of data; occurrence options are every time, n times, and true for a duration. (verified: https://learn.microsoft.com/en-us/fabric/real-time-intelligence/data-activator/activator-detection-conditions (updated 2026-05-04))
- Actions include email, Teams message (individual, group chat, channel), Power Automate and running Fabric items such as pipelines and notebooks. (verified: https://learn.microsoft.com/en-us/fabric/real-time-intelligence/data-activator/rule-actions (updated 2026-05-11))
- Up to 10,000 incoming events per second per rule, after which the rule stops; 30 email or Teams messages per rule per recipient per hour. (verified: https://learn.microsoft.com/en-us/fabric/real-time-intelligence/data-activator/activator-limitations (updated 2026-04-17))
- Late events are dropped by default; the wait time for late-arriving events defaults to two minutes and adds latency. (verified: https://

learn.microsoft.com/en-us/fabric/real-time-intelligence/data-activator/activator-latency (updated 2026-04-30))

Track C best-practice checklist

☐ Check the status (GA or preview) of every Fabric feature before you depend on it.

☐ One copy of each dataset. Use shortcuts or mirroring before you copy.

☐ Lakehouse for Spark and files. Warehouse for T-SQL writes and multi-table transactions.

☐ The SQL analytics endpoint is read-only for table data. Write through Spark or a Warehouse.

☐ Semantic models use star schemas. DAX measures state their filter context.

☐ Direct Lake models are checked for fallback to DirectQuery.

☐ Every Copilot output is read, run on known data and compared against a second method.

☐ Workspaces map to domains. Access uses roles and OneLake security, not item sharing.

☐ Items live in Git. Changes move through deployment pipelines.

☐ Capacity use is monitored, and heavy jobs are scheduled off peak.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

C7-L1 Eventstream, events in, routed out

C7-L1 exercise 1

Hints. (1) Which format does a lakehouse use for tables? (2) Delta. (3) Eventhouse is a different destination.

Answer. b) Delta Lake tables Because: r1) The lakehouse destination converts real-time events to Delta Lake format in lakehouse tables.

Lakehouse destinations write Delta tables. Eventhouse destinations write KQL tables.

Common wrong answers.

- `a`: Events become Delta tables, which Spark, SQL and Direct Lake can read.
- `c`: Events become Delta tables.
- `d`: KQL tables live in an eventhouse, a different destination.
- `b|r2`: Right answer, wrong reason. Eventhouse destinations store data in KQL databases.

C7-L1 exercise 2

Hints. (1) Writing to an item needs which role? (2) Lesson C1-L3. (3) Contributor.

Answer. b) Contributor or higher Because: r1) Microsoft Learn requires Contributor or higher on the destination workspace for eventhouse, lakehouse and Activator destinations.

Contributor or higher on the destination workspace.

Common wrong answers.

- `a`: Viewers cannot write to items (lesson C1-L3).
- `c`: Admin is more than needed.
- `d`: Access to the destination workspace is required.
- `b|r2`: Right answer, wrong reason. Contributor applies to the workspace where the role is granted.

C7-L1 exercise 3

Hints. (1) Source first. (2) Protect the raw data before filtering. (3) Verify at the end.

Answer.

1. Connect the source
2. Route every event to a raw table
3. Create derived streams with filters for each consumer
4. Connect each derived stream to its destination
5. Compare event counts at source and destinations

Source, raw landing, derived streams, destinations, then a count check.

Common wrong answers.

- Store the raw stream first, so nothing is lost while you design filters.
- Counts can only be compared once destinations are connected and receiving events.

C7-L1 exercise 4

Hints. (1) Start with telemetry_raw for every event. (2) Check for a missing key before reading it. (3) Add fault destinations after the raw one.

Answer.

```python
def route(event: dict[str, object]) -> list[str]:
    """Destinations for one telemetry event. Every event is kept raw.

    Args:
        event: Parsed event.

    Returns:
        Destination names in routing order.
    """
    destinations = ["telemetry_raw"]
    if "event_type" not in event:
        destinations.append("quarantine")
    elif event["event_type"] == "fault":
        destinations += ["activator_faults", "bronze.vehicle_faults"]
    return destinations
```

Raw first, always. Malformed events are kept and flagged, not lost.

C7-L1 exercise 5

Hints. (1) Which source captures every change? (2) Which store answers queries in seconds? (3) Which store does Spark read?

Answer.

- The CDC source streams inserts, updates and deletes after an initial snapshot.
- An eventhouse serves low-latency KQL queries for dashboards.
- A lakehouse destination writes Delta tables for Spark.

One CDC source, two destinations, each suited to its consumer.

Common wrong answers.

- source = `poll`: Polling misses deletes and adds load. CDC streams every change.
- live = `lakehouse`: Lakehouse tables suit batch analysis. Live dashboards query an eventhouse with KQL.
- models = `custom`: Spark reads Delta tables in a lakehouse directly.

C7-L2 Eventhouse and KQL for time-series questions

C7-L2 exercise 1

Hints. (1) Does take sort? (2) Is its result consistent? (3) Use top for ordered results.

Answer. c) 10 arbitrary rows, possibly different on each run Because: r1) take guarantees nothing about which records it returns unless the data is sorted.

take is for browsing. top is for answers.

Common wrong answers.

- `a`: Use top 10 by event_time desc for the most recent rows.
- `b`: Ingestion order is not guaranteed either.
- `d`: take runs without error. It just promises nothing about which rows.
- `c|r2`: Right answer, wrong reason. There is no fixed seed. Results can change between runs.

C7-L2 exercise 2

Hints. (1) Which rows does summarize group? (2) Are there rows in those bins? (3) No.

Answer. b) No rows for those bins Because: r1) summarize groups existing rows. Bins with no rows do not appear. make-series fills them.

Charts draw a line straight across the gap. Use make-series with a default to show the outage.

Common wrong answers.

- `a`: That is make-series with default = 0. summarize only returns groups that exist.
- `c`: No rows are created at all.
- `d`: Empty bins are not an error.
- `b|r2`: Right answer, wrong reason. There are no rows to count, so no group is formed.

C7-L2 exercise 3

Hints. (1) A function for 'one hour before now'. (2) The operator that sorts and limits. (3) `ago` and `top`.

Answer.

```
1 telemetry_raw
2 | where event_time > ago(1h)
3 | top 10 by event_time desc
```

`ago(1h)` sets the window. `top 10 by event_time desc` returns the latest ten.

Common wrong answers.

- g2 = `take`: take does not accept a sort and returns arbitrary rows.

C7-L2 exercise 4

Hints. (1) Filter with where and ago first. (2) make-series fills missing steps with the default. (3) ... default = 0 on event_time step 1h by vehicle_id.

Answer.

```
1 telemetry_raw
2 | where event_time > ago(24h)
3 | make-series avg_rpm = avg(rpm) default = 0 on event_time step 1h by
    vehicle_id
```

make-series returns one array per vehicle with every hour present, so gaps show as zeros.

C7-L2 exercise 5

Hints. (1) Who needs the raw data, and for how long? (2) Which period is actually queried? (3) Aggregates are small.

Answer.

- Keep raw data as long as audits need, and no longer: 90 days.
- Cache the 30 days people query, to balance speed and cost.
- Keep small aggregates for years, where trends need them.

Retention follows need, cache follows query patterns, and aggregates outlive raw data.

Common wrong answers.

- raw_ret = `keep_default`: Ten years of raw telemetry costs storage nobody uses. Set retention from the need.
- raw_ret = `d7`: Audits need 90 days. Data past retention is removed automatically.
- raw_cache = `c90`: Caching data nobody queries raises cost without speeding up real queries.

C7-L3 Activator, rules that act on events

C7-L3 exercise 1

Hints. (1) Is this a change or a state condition? (2) How many times does the value cross 100 from below? (3) Once.

Answer. a) Once Because: r1) Increases above is a change condition. It fires when the value crosses the threshold from below.

Change conditions fire on transitions. State conditions fire on every matching event.

Common wrong answers.

- `b`: That is Is greater than, a state condition that fires on each matching event.
- `c`: The crossing from below is the trigger. No decrease is needed.
- `d`: Change conditions do not repeat on a timer.
- `a|r2`: Right answer, wrong reason. There is no daily limit of one. The rule fires once because there is one crossing.

C7-L3 exercise 2

Hints. (1) What arrives from a silent sensor? (2) Nothing. Which condition works on absence? (3) Heartbeat.

Answer. b) No presence of data Because: r1) No presence of data fires when no new events arrive within a set time. Other conditions need an event to evaluate.

Every value-based condition needs an event. Only the heartbeat condition sees silence.

Common wrong answers.

- `a`: No event arrives, so there is no value to compare.
- `c`: Becomes false needs a new event with the value false.
- `d`: Changes needs a new event with a different value.
- `b|r2`: Right answer, wrong reason. A silent sensor sends nothing at all.

C7-L3 exercise 3

Hints. (1) Which column identifies one freezer? (2) The occurrence option reads 'When it has been ... for'. (3) `freezer_id` and `true`.

Answer.

```
Object: freezer, keyed by freezer_id
Condition: temperature Is greater than -15
Occurrence: When it has been true for 10 minutes
```

Key by the object. The occurrence turns a state condition into one alert for a sustained problem.

Common wrong answers.

- g1 = `event_id`: Every event has its own ID, so no state carries across readings. Key by the freezer.

C7-L3 exercise 4

Hints. (1) Keep a dict from object_id to 'was above'. (2) Fire when the value is above and the object was not. (3) Update the object's state after every event.

Answer.

```python
def activations(events: list[tuple[str, float]], threshold: float) -> list[int]:
    """Indexes of events where an object's value crosses above a threshold.

    Args:
        events: (object_id, value) pairs in arrival order.
        threshold: Value that must be exceeded.

    Returns:
        Event indexes where the object moved from at-or-below to above.
    """
    above: dict[str, bool] = {}
    fired: list[int] = []
    for index, (object_id, value) in enumerate(events):
        is_above = value > threshold
        if is_above and not above.get(object_id, False):
            fired.append(index)
```

```
        above[object_id] = is_above
    return fired
```

Per-object state is what makes the rule mean "this oven crossed 80", not "some reading somewhere crossed 80".

C7-L3 exercise 5

Hints. (1) Which ID identifies one van? (2) How many readings arrive per minute? (3) Which condition works without events?

Answer.

- Key by van_id, so each van keeps its own state.
- A sustained state condition sends one message per incident, well under the 30 per rule per recipient per hour limit.
- A heartbeat rule detects silence, which no value-based condition sees.

Per-object keys, sustained conditions for noisy streams, and a heartbeat for silence.

Common wrong answers.

- object = `fleet`: One object mixes 400 vans. A cool reading from one van clears a hot one.
- hot = `every`: At one reading every 5 seconds, a hot van sends 12 messages a minute and hits the 30-per-hour limit in under 3 minutes.
- silent = `speed_zero`: A parked van reports speed 0. A silent van reports nothing.